

1 Qué es

Una sola app para **Android e iPhone** que coordina **3 celulares sin internet** para fotografiar ambos lados de cada hilera de vid. Cada disparo guarda dos fotos con su contexto: **lote, hilera, lateral, marcador y GPS**. Después del login se elige qué será el celular: **Controlador, Cámara 1 o Cámara 2**.

25

pantallas

20

tablas en SQLite

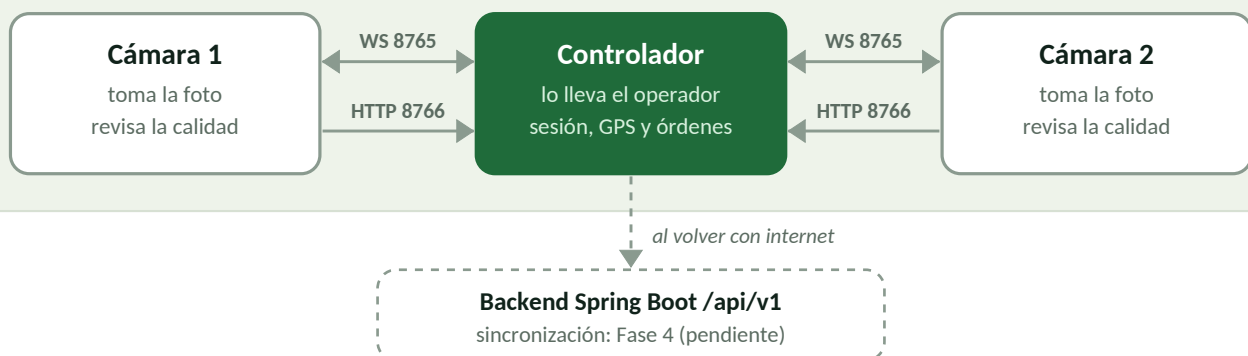
65

pruebas (64 pasan)

20 760

líneas de TypeScript

Wi-Fi local del campo, sin internet (por ejemplo, hotspot del controlador)



WS 8765: órdenes, estados y confirmaciones (ACK) en ambos sentidos, con señal de vida cada 2 s. **HTTP 8766:** cada foto va de la cámara al controlador, verificada con MD5.

2 Tecnologías y para qué sirven

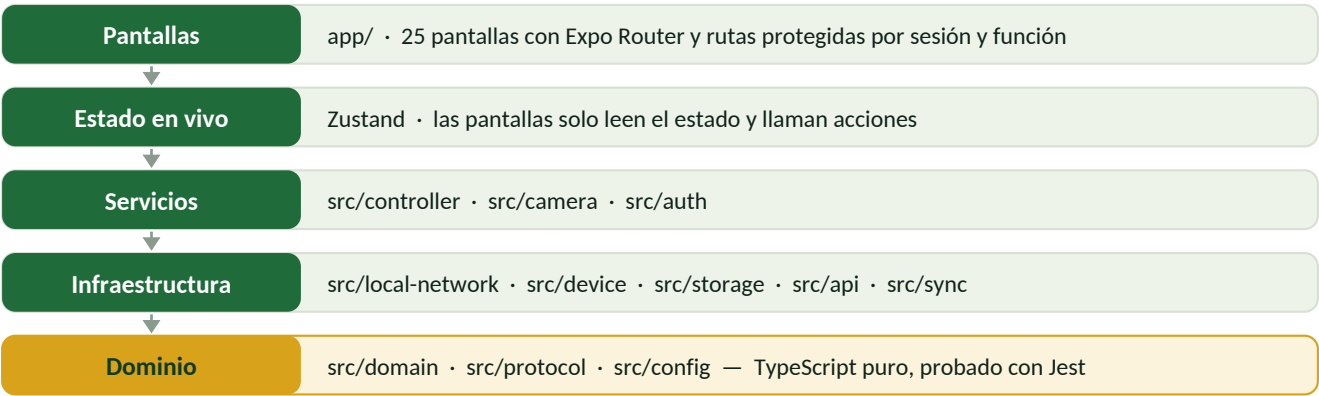
Tecnología	Para qué se usa
TypeScript (modo estricto)	Lenguaje de todo el código; detecta errores antes de ejecutar
React Native 0.86 + React 19	Una sola base de código para Android e iOS
Expo SDK 57	Plataforma con los módulos oficiales: cámara, GPS, SQLite, sensores
Expo Go	Probar en el celular sin compilar. No abre servidor de red: ahí el controlador usa cámaras simuladas
EAS Build → APK	Compila en la nube el instalable de Android. El controlador real necesita el APK
Expo Router	Navegación: cada archivo de <code>app/</code> es una pantalla, con rutas protegidas
SQLite • SecureStore • File System	Datos dentro del celular: registros y colas • tokens y verificador • fotos
expo-camera, -location, -sensors	Foto y lectura del QR • GPS • detectar si el celular está quieto
Image Manipulator + jpeg-js	Calidad de la foto (exposición y nitidez), sin inteligencia artificial
Zustand • Zod	Pantallas que se actualizan solas • mensajes válidos entre celulares
react-native-tcp-socket	Convierte al controlador en servidor WebSocket + HTTP (solo en APK)
@noble/hashes • expo-crypto	Login sin internet sin guardar la contraseña (PBKDF2) • identificadores y MD5
Jest • ESLint • Git	Pruebas automáticas • revisión de estilo • control de versiones

3 Cómo funciona

- 1 Login con internet; después, hasta **7 días sin internet** con el mismo usuario.
- 2 Elegir la función del celular y aceptar los permisos (ubicación o cámara).
- 3 El controlador crea la sesión (**MANUAL** o **AUTOMÁTICO**) y muestra un **QR**; cada cámara lo escanea y queda vinculada.
- 4 **Prueba corta** con ambas cámaras → nueva **pasada** (lote, hilera, lateral y marcador).
- 5 Captura: el controlador ordena, cada cámara toma la foto, revisa su calidad y la envía. Si sale mala se repite solo esa cámara.
- 6 Al cerrar la sesión, los datos quedan en una **cola** para enviarse a la web (Fase 4).

4 Arquitectura por capas

Cada capa solo usa a las de abajo: las pantallas muestran el estado y llaman servicios, nunca tocan la base de datos ni la red. Las reglas de negocio no dependen de la interfaz, por eso se prueban con Jest.



Carpetas

Carpeta	Qué hace hoy	A futuro
app/	25 pantallas: login, función, controlador (12), cámara (3)	Pantallas de sincronización y ciclos
src/ui/	Colores, textos, componentes, sonidos y vibración	—
src/config/	Parámetros (tiempos, umbrales, puertos) y .env	URL del backend real
src/domain/ src/protocol/	Reglas de negocio, estados y mensajes entre celulares	Ciclo vigente desde el backend
src/controller/	Sesión, pasadas, secuencias, QR y recepción de fotos	Avance de todos los controladores
src/camera/	Tomar, revisar y enviar la foto, con reintentos	—
src/local-network/	Red real (tcp/), cliente de cámara (client/), simulador	Alternativa nativa si falla TCP
src/device/	Cámara, GPS, sensores, batería, espacio, permisos, calidad	—
src/storage/	SQLite (20 tablas), archivos de fotos y SecureStore	Copia local; la verdad en PostgreSQL
src/auth/	Login con y sin internet, tokens y validaciones	Igual; solo cambia la API
src/api/ src/sync/	Contrato /api/v1, backend simulado y reintentos	Backend real y envío (Fase 4)
src/diagnostics/	Registro de eventos, incidencias y diagnóstico	Incidencias al backend
__tests__/ docs/	65 pruebas · decisiones técnicas (ADR) y archivo maestro	—
Raíz	app.json app.eas.json APK · package.json dependencias	—

5 Estado actual (versión 0.2.0)

Fase	Contenido	Estado
0	Base del proyecto, base de datos local y pruebas	● Hecha
1	Acceso: login con y sin internet, registro, contraseñas, función del celular, permisos	● Hecha
2	Vinculación por QR, red local, prueba corta, simulador de cámaras	● Hecha en código falta prueba con 3 Android
3	Monitoreo: pasadas, captura MANUAL/AUTOMÁTICO, calidad, repetición y cierre	● Hecha en código falta prueba en campo
4	Sincronización con la web y la base de datos central	● Pendiente en la app dice "Pendiente"

6 Qué falta implementar

- **Backend (Spring Boot, API /api/v1):** aún no existe; lo desarrolla otro integrante. La app usa un **backend simulado** con el mismo contrato y cuentas de prueba; para conectarlo solo se cambia la URL en `.env`.
- **Base de datos central (PostgreSQL) y fotos (Amazon S3):** sin conectar. Hoy todo vive en **SQLite** dentro de cada celular; la app nunca se conectará directo a ellas, solo a través del backend.
- **Fase 4, sincronización:** la cola ya se llena al cerrar cada sesión; falta el código que la envía en orden (sesión → pasadas → secuencias → fotos → incidencias).
- **Ciclos y cobertura:** código a medias (bloquear lo ya monitoreado en cada ciclo); hoy da 14 errores de tipos y 1 prueba fallida.
- **Pruebas reales:** vinculación con 3 Android usando el APK, calibración de umbrales en campo y códigos reales de segmentos y marcadores.
- **Fuera de la app:** la web de revisión (Angular + Leaflet) y la IA de detección de plagas (YOLO) son otros componentes del sistema.